

Programming Assignment

Lightweight Backbone Design for Object Detection

Edge Computing · Pascal VOC 2012 · Faster R-CNN · Trained from Scratch

1. Overview

In this assignment you will design and train a **lightweight custom CNN backbone** from scratch for object detection on the **Pascal VOC 2012** dataset. Your backbone will be plugged into a fixed Faster R-CNN detection head — the same head used by the instructor's provided baseline — and evaluated on mean Average Precision at IoU threshold 0.50 (mAP@50), model size, and inference speed.

Baseline to beat: A pretrained MobileNetV3-Small backbone, which achieves **mAP@50 $\approx$ 0.457** and an average inference latency of **$\sim$ 10 ms / image** on a T4 GPU. Your goal is to match or exceed this mAP while keeping your backbone small and fast, **using only randomly initialized weights**.

2. Dataset Structure

The dataset used in this assignment is based on **Pascal VOC 2012**, a standard benchmark for object detection containing 20 object categories. For this assignment, the data has been reorganized into two top-level folders: a **train folder** (images with annotations) and a **test folder** (images only, no labels). The train/validation split is handled entirely inside the code using image ID list files — you do not need to move or reorganize any files.

2.1 Directory Layout

```
dataset/
├── JPEGImages/      - all images (.jpg) - both train and test
├── Annotations/     - bounding box labels in Pascal VOC XML format
└── ImageSets/Main/
    ├── train.txt     - image IDs for the annotated training images
    └── test.txt      - image IDs for the hidden test images (no annotations)
```

2.2 Train and Validation Split

All annotated images live inside the `train/` folder. The split into training and validation subsets is done **entirely in code at runtime** — you do not need to create or move any files. The notebook reads `train.txt`, shuffles the image IDs with a fixed seed, and holds out **10% as validation** (controlled by `CFG.VAL_RATIO = 0.1`):

```
# From the notebook — done automatically, do not change
random.seed(CFG.SEED)
random.shuffle(original_train_ids)

n_val = int(len(original_train_ids) * CFG.VAL_RATIO) # 10%

val_ids      = sorted(original_train_ids[:n_val])    # validation set
train_ids_new = sorted(original_train_ids[n_val:])    # training set
```

Both subsets are passed directly as lists of image IDs into `VOCDataSet`. Both have ground-truth XML annotations, so you can compute `mAP@50` on the validation set locally before uploading to Kaggle.

2.3 Test Set

The `test/JPEGImages/` folder contains the hidden evaluation images. There are **no annotation files** for the test set — you will never see the ground-truth labels. Your workflow for Part 1 submission is:

1. Load your trained model and set it to eval mode.
2. Run inference on every image in `test/JPEGImages/`.
3. Collect all predicted boxes, scores, and class labels.
4. Save the predictions in the format required by the Kaggle competition page.
5. Upload the file to the leaderboard — your `mAP@50` is computed and shown automatically.

2.4 Object Categories

The dataset covers 20 object categories. Class indices match the order defined in `VOC_CLASSES` in the notebook (index 0 is reserved for background and is never predicted):

Index	Class	Index	Class
1	aeroplane	11	diningtable
2	bicycle	12	dog
3	bird	13	horse
4	boat	14	motorbike
5	bottle	15	person
6	bus	16	pottedplant
7	car	17	sheep
8	cat	18	sofa
9	chair	19	train
10	cow	20	tvmonitor

3. Provided Code Structure

The notebook you have been given implements a complete training, evaluation, and submission pipeline. You do not need to change any part of the code except the StudentNet class. The sections are organized as follows:

object-detection-2026.ipynb	
Import Libraries	- standard PyTorch / torchvision / pandas imports
Kaggle GPU Safety Check	- selects CUDA device and AMP flag
Experiment Configuration (CFG)	- all hyperparameters in one class
VOC Class Names & Seed	- 20-class label map, reproducibility, XML parser
Data Augmentation	- random horizontal flip (bbox-aware)
Train/Val Split	- 90/10 random split from train.txt at runtime
VOCDataset	- labelled dataset (train + val), accepts ID list
VOCTestDataset	- unlabelled test dataset (images + image_id only)
Batch Collation	- collate_fn (train/val) and collate_fn_test
StudentNet_Pretrained	- MobileNetV3-Small baseline (reference only)
StudentNet <<<	YOUR EDITABLE CLASS
DetectorBackbone	- wraps StudentNet with 1x1 adapter conv
Parameter Breakdown Utility	- prints param counts per component
Faster R-CNN Model Builder	- assembles the full detector
DataLoaders	- train / val / test loaders
Optimizer & AMP Setup	- AdamW + GradScaler
Training Loop	- best checkpoint saving per epoch
Validation Loss	- loss computed on val set each epoch
Inference Latency	- CUDA-synchronized timing per image
mAP@50 Evaluation	- lightweight VOC-style precision-recall on val set
Save Final Weights	- persists model to /kaggle/working
File Submission	- runs inference on test set, saves submission.csv
ONNX Conversion	- exports model to model.onnx (legacy exporter)

4. Key Components Explained

4.1 Experiment Configuration (CFG)

All training hyperparameters live in one class so you can change them in one place. The most relevant fields for this assignment are:

- **USE_PRETRAINED_BASELINE** — set to `False` to use your custom backbone; `True` runs the pretrained MobileNetV3-Small baseline
- **VAL_RATIO = 0.1** — 10% of the training images are held out as a local validation set; the remaining 90% are used for training
- **TEST_SPLIT = "test"** — points to the hidden test set used for Kaggle submission; do not change this
- **EPOCHS = 1, BATCH_SIZE = 16, LR = 1e-4** — you may tune these (increase epochs for a real training run)
- **MIN_SIZE = MAX_SIZE = 384** — images are resized to 384x384 before the backbone
- **FIXED_OUT_CHANNELS = 128** — the standardized channel size that the RPN and detection head always expect (explained in detail below)

- **SUBMISSION_PATH** — path where `submission.csv` is saved after running the File Submission cell

Understanding `FIXED_OUT_CHANNELS` and `self.out_channels`

This is one of the most important relationships to understand before designing your backbone. The pipeline uses a **two-stage channel interface** that completely decouples your backbone from the detection head:

```
# Inside DetectorBackbone:
self.adapter = nn.Conv2d(
    student_backbone.out_channels,    # reads YOUR backbone's output channels
    fixed_out_channels,               # always 128 (from CFG.FIXED_OUT_CHANNELS)
    kernel_size=1
)
```

The `DetectorBackbone` wrapper inserts a **1x1 adapter convolution** between your backbone and the RPN. This adapter reads `student_backbone.out_channels` — a plain integer attribute you declare inside `StudentNet` — and projects your feature map to exactly `FIXED_OUT_CHANNELS = 128` channels. The RPN and detection head then always operate on a 128-channel feature map, regardless of what your backbone produces internally.

This means:

- **Your backbone can output any number of channels** (e.g. 32, 64, 256). You are free to choose whatever suits your architecture.
- **The adapter bridges the gap automatically.** For example, the provided pretrained baseline (MobileNetV3-Small) outputs 576 channels. The adapter becomes a `Conv2d(576 -> 128)`, and the detection head never sees 576.
- **`self.out_channels` must be declared correctly.** This integer must match your last conv layer's actual output channels exactly. If they disagree, the adapter will be initialized with the wrong size and the model will crash at runtime.

Example: If your last conv layer outputs 64 channels, you must set `self.out_channels = 64`. The adapter will then be `Conv2d(64 -> 128)`, and everything downstream works automatically. You do not need to touch `FIXED_OUT_CHANNELS` at all.

4.2 The `StudentNet` Class (Your Workspace)

This is the only class you need to modify. It defines a feature extractor that takes a 3-channel image tensor and returns a feature map. The default implementation is a minimal 3-layer conv stack:

```

class StudentNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),

            nn.Conv2d(32, 64, 3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),

            nn.Conv2d(64, 128, 3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
        )
        self.out_channels = 128 # MUST match the last layer's output channels

    def forward(self, x):
        return self.features(x)

```

⚠ Important constraint: You must keep `self.out_channels` set to whatever your last conv layer actually outputs. The `DetectorBackbone` wrapper reads this attribute to set up the 1×1 adapter convolution that connects your backbone to the RPN.

4.3 DetectorBackbone Wrapper

You do **not** need to modify this class. It is the glue between your `StudentNet` and the Faster R-CNN detection head, and it works in two steps:

6. **Runs your backbone:** `x = self.features(x)` — passes the input image tensor through your `StudentNet` and produces a spatial feature map of shape `[B, C, H', W']`, where `B` is the batch size, `C` is your backbone's output channel count (whatever you set `self.out_channels` to), and `H'`, `W'` are the spatial dimensions after all stride-2 downsampling steps.
7. **Applies the 1x1 adapter:** `x = self.adapter(x)` — a single `Conv2d(C, 128, kernel_size=1)` pointwise convolution that projects your backbone's feature map from `C` channels to exactly `FIXED_OUT_CHANNELS = 128` channels. This projection adds very few parameters (`C × 128` weights) and no spatial distortion, making it a cheap but effective interface layer. The result is wrapped in an `OrderedDict` with key `"0"` as required by the Faster R-CNN API.

This is why the pretrained baseline (MobileNetV3-Small, 576 output channels) and a custom lightweight backbone (e.g. 64 output channels) can both plug into the **exact same detection head** without any changes. The adapter acts as a standardized interface — your backbone's internal design is entirely your own, as long as `self.out_channels` is declared correctly.

4.4 Faster R-CNN Detection Head

The detection head is fixed and assembled by `build_model()`. You do not need to modify any of it. It receives the 128-channel feature map from `DetectorBackbone` and runs three stages to produce final bounding box predictions:

- **Region Proposal Network (RPN)** — a small convolutional network that slides over the 128-channel feature map and scores a set of predefined **anchor boxes** at every spatial location. Anchors cover five scales (32, 64, 128, 256, 512 pixels) and three aspect ratios (0.5, 1.0, 2.0), giving 15 anchors per location. The RPN outputs two things: an objectness score (foreground vs. background) and a box offset to refine each anchor. The top-scoring refined anchors are kept as **region proposals** and passed to the next stage.
- **RoI Align** — each region proposal has a different size and position on the feature map. RoI Align crops and resizes each proposal's feature region into a fixed 7×7 grid using bilinear interpolation (sampling ratio 2). This produces a uniform $[N, 128, 7, 7]$ tensor for all proposals, regardless of their original size, where N is the number of proposals in the batch.
- **Two-layer MLP head** — the $7 \times 7 \times 128 = 6,272$ -dimensional RoI feature is flattened and passed through two fully connected layers ($6272 \rightarrow 1024 \rightarrow 1024$). This produces a compact, class-agnostic representation for each proposal.
- **FastRCNN Predictor** — two parallel linear layers run on the 1024-dimensional output: a **classification head** ($1024 \rightarrow 21$ logits, one per class including background) and a **box regression head** ($1024 \rightarrow 84$ offsets, four per class). These produce the final class label and refined bounding box for each proposal.

Why does this matter for your backbone design? The RPN slides over your backbone's spatial feature map. If your backbone downsamples too aggressively (e.g. total stride 32x or more on a 384x384 input), the feature map becomes very small and small objects may be missed entirely. A total stride of 8x produces a 48x48 feature map; 16x gives 24x24. Aim for a total stride between 8x and 16x for good detection coverage.

4.5 Evaluation — Part 1: Detection Accuracy

For Part 1 of the assignment, the only metric that matters is **mAP@50**. There are no constraints on model size, number of parameters, or architecture style. You have complete freedom in how you design your backbone — the only rules are that weights must be trained from scratch and the `StudentNet / DetectorBackbone` interface must be respected.

- **mAP@50 (sole metric for Part 1)** — mean Average Precision across all 20 VOC classes at an IoU threshold of 0.50. For each class, a predicted box is counted as a true positive only if it overlaps a ground-truth box by at least 50% ($\text{IoU} \geq 0.5$) and has the correct class label. The precision-recall curve for each class is summarized into a single AP value (area under the curve). mAP is the mean of these 20 AP values. A score of 1.0 is perfect; the pretrained baseline achieves approximately **0.457**. Per-class AP values are also printed so you can see where your model is strong or weak.

No architecture constraints in Part 1. You are free to use any number of layers, channels, or block types. A larger model that trains well is perfectly acceptable at this stage. Efficiency will become important in Part 2, when your model must run on a Jetson Orin.

4.6 Evaluation — Part 2: Edge Deployment

In Part 2, you will take the trained model from Part 1 and deploy it on an **NVIDIA Jetson Orin** embedded platform. This involves two conversion steps and a final latency measurement on the target hardware.

Step 1: Export to ONNX

ONNX (Open Neural Network Exchange) is a standard format for representing deep learning models independently of the training framework. You will export your trained PyTorch model to an `.onnx` file using `torch.onnx.export()`. This file encodes the full computation graph of your model and can be consumed by TensorRT on the Jetson Orin.

```
import torch

# 1. Set model to eval mode and move to CPU
# (ONNX export is more stable on CPU)
model.eval()
model.to("cpu")

# 2. Faster R-CNN expects a LIST of tensors, not a batched tensor
dummy_input = [torch.randn(3, 384, 384)]

# 3. Export using the legacy TorchScript-based exporter
with torch.no_grad():
    torch.onnx.export(
        model,
        (dummy_input,),          # wrap the list in a tuple
        "model.onnx",
        opset_version=11,        # opset 11 is required for Faster R-CNN
        dynamo=False,           # REQUIRED — see note below
        do_constant_folding=True,
        input_names=["input"],
        output_names=["boxes", "labels", "scores"],
    )

print("Exported to model.onnx")
```

Why dynamo=False is required for Faster R-CNN

PyTorch 2.x introduced a new dynamo-based ONNX exporter that uses `torch.export` to trace the model symbolically. This fails on Faster R-CNN because the RPN's Non-Maximum

Suppression (NMS) operation contains a data-dependent branch (`if boxes.numel() == 0`) whose value depends on the actual number of proposals at runtime, which is unknown at trace time. Setting `dynamo=False` forces the legacy TorchScript-based exporter, which traces with concrete values and handles this correctly.

Step 2: Convert to TensorRT

TensorRT is NVIDIA's high-performance deep learning inference library. It takes the ONNX graph and compiles it into a highly optimized **TensorRT engine** that is specifically tuned for the target GPU hardware. On a Jetson Orin, this means layer fusion, kernel auto-tuning, and optional FP16 or INT8 quantization — all of which dramatically reduce latency compared to running the raw PyTorch model.

The conversion is done on the Jetson Orin itself using `trtexec`, NVIDIA's command-line TensorRT tool:

```
# On the Jetson Orin terminal:
trtexec --onnx=model.onnx \
        --saveEngine=model.trt \
        --fp16 \
        --workspace=2048
```

Note: For this part, the student should come to the lab to access to Jetson Orin board

Step 3: Measure Latency on Jetson Orin

Once the TensorRT engine is built, you will measure **end-to-end inference latency** on the Jetson Orin by running the engine on a batch of validation images and recording wall-clock time per image. This is the latency number you will report. Unlike the Kaggle T4 measurement in the training notebook, this reflects **real edge deployment conditions** — the Jetson Orin has a much smaller power envelope and different memory bandwidth than a data center GPU, making efficient backbone design directly visible in the numbers.

Why does Part 1 freedom matter here? A model that achieves high mAP@50 in Part 1 but is very large or slow will produce poor latency numbers in Part 2. The design choices you make in Part 1 — depth, channel widths, block types — will directly impact your Part 2 results. This is intentional: the assignment mirrors the real-world trade-off between accuracy and efficiency in edge AI systems.

5. Your Assignment

Your task is split into two sequential parts. **Complete Part 1 before moving to Part 2**, as the trained model from Part 1 is the input to the deployment pipeline.

5.1 Part 1 — Design and Train

Redesign the **StudentNet** class and train your model on Pascal VOC 2012. The only requirements are:

8. **Train entirely from scratch** — no pretrained weights may be loaded anywhere in your model.
9. **Outperform the baseline** — achieve a mAP@50 higher than 0.457 on the VOC 2012 validation set.
10. **Respect the interface** — your StudentNet must declare `self.out_channels` correctly and return a single feature map tensor from `forward()`

No other constraints apply in Part 1. You may use any architecture, any number of parameters, any channel widths, any block types. Larger models are fine at this stage.

5.2 Part 2 — Deploy on Jetson Orin

Take your best trained model from Part 1 and deploy it on the Jetson Orin following the pipeline in Section 3.6:

11. **Export to ONNX** using `torch.onnx.export()`
12. **Convert to TensorRT** using `trtexec` on the Jetson Orin
13. **Measure and report latency** — average inference time per image on the Jetson Orin

What you are allowed to change

- **StudentNet class** — full freedom to redesign the architecture, no constraints on size or style
- **CFG hyperparameters** — you may tune LR, batch size, epochs, image size, and `FIXED_OUT_CHANNELS`
- **DetectionTransform** — you may add more augmentations (color jitter, random crop, etc.)

What you must NOT change

- **Pretrained weights** — do not load weights from `torchvision.models` or any external checkpoint
- **DetectorBackbone, build_model, evaluate_map_placeholder** — these must remain unchanged
- **Dataset and data splits** — use the standard Pascal VOC 2012 train/val split

6. Design Hints

Building a competitive backbone from scratch is challenging. The following techniques are commonly used in efficient network design and are recommended as starting points. You are not required to use all of them.

6.1 Depthwise Separable Convolutions

A standard $k \times k$ convolution with C_{in} input channels and C_{out} output channels costs $k^2 \times C_{in} \times C_{out}$ multiply-adds per spatial location. A **depthwise separable convolution** splits this into two steps:

- **Depthwise conv** — one filter per input channel (spatial mixing only); cost: $k^2 \times C_{in}$
- **Pointwise conv (1×1)** — mixes channels; cost: $C_{in} \times C_{out}$

This reduces computation by a factor of roughly 8–9× for 3×3 kernels, with only a small accuracy penalty. In PyTorch:

```
# Depthwise separable block
nn.Conv2d(C, C, 3, padding=1, groups=C, bias=False), # depthwise
nn.BatchNorm2d(C),
nn.ReLU(inplace=True),
nn.Conv2d(C, C_out, 1, bias=False), # pointwise
nn.BatchNorm2d(C_out),
nn.ReLU(inplace=True),
```

6.2 Bottleneck / Inverted Residual Blocks

A **bottleneck block** squeezes channels with a 1×1 conv, applies a cheap 3×3 conv in the narrow space, then expands back. An **inverted residual block** (used in MobileNetV2/V3) does the opposite: it **expands** first, applies a depthwise 3×3, then **projects back down**. Residual connections (when input and output shapes match) allow gradients to flow cleanly, enabling deeper networks without degradation.

```
# Inverted residual block sketch
class InvResBlock(nn.Module):
    def __init__(self, in_c, expand_c, out_c, stride=1):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_c, expand_c, 1, bias=False), nn.BatchNorm2d(expand_c),
            nn.ReLU6(inplace=True),
            nn.Conv2d(expand_c, expand_c, 3, stride=stride, padding=1,
                groups=expand_c, bias=False),
            nn.BatchNorm2d(expand_c), nn.ReLU6(inplace=True),
            nn.Conv2d(expand_c, out_c, 1, bias=False), nn.BatchNorm2d(out_c),
        )
        self.use_skip = (stride == 1 and in_c == out_c)
    def forward(self, x):
        return x + self.conv(x) if self.use_skip else self.conv(x)
```

6.3 Progressive Channel Reduction

A common mistake is to use large channel widths throughout the network. Instead, **start narrow and only expand where necessary**. For a backbone feeding into a detection head, a channel schedule such as $3 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$ is often sufficient. Avoid going above 256 channels in a lightweight model — the parameter count grows quadratically with channel width in pointwise convolutions.

- **Wider $\neq$ better**: doubling channels quadruples parameters in conv layers.
- **Depth helps more than width**: more stages with fewer channels generally gives a better accuracy/cost trade-off.
- **Match spatial stride to task**: for a 384×384 input feeding Faster R-CNN, a total stride of $8\times$ (three stride-2 layers) keeps the feature map large enough for small object detection.

6.4 Strided Convolutions Instead of Pooling

Replace `MaxPool2d` or `AvgPool2d` with **stride-2 convolutions**. This gives the network a **learnable downsampling** operation, which typically improves detection accuracy because the network can learn which information to preserve rather than using a fixed pooling function.

```
# Instead of: nn.MaxPool2d(2)
# Use:       nn.Conv2d(C, C, 3, stride=2, padding=1, bias=False)
#           followed by BatchNorm and activation
```

7. Deliverables

The assignment uses a Kaggle competition format for Part 1 evaluation. You will run inference on a hidden test set, submit your predictions to the Kaggle leaderboard, and your `mAP@50` score will be computed and displayed automatically. You do not have access to the test labels.

7.1 Part 1 — Kaggle Leaderboard Submission

After training, run your model in inference mode on the provided **test dataset** and save your predictions to a **CSV file** in the format specified below. Upload this file to the course Kaggle competition page — your `mAP@50` will appear on the leaderboard automatically.

Prediction File Format

The submission file is generated automatically by the **File Submission** cell at the end of the notebook. It calls `export_submission_csv()`, which runs inference on the test set and saves `submission.csv` to `/kaggle/working/`. You simply download this file and upload it to the Kaggle competition page.

The CSV has four columns:

```
id, image_id, PredictionString, Usage

# id          : row index (0, 1, 2, ...)
# image_id    : image filename without extension (e.g. 2007_000032)
# PredictionString: all detections for this image as a flat space-separated
string:
#             label score xmin ymin xmax ymax  (repeated per detection)
#             e.g. "15 0.981234 104.0 62.0 237.0 334.0 1 0.741234 0.0 80.0
110.0 210.0"
#             Images with no detections use the placeholder: __EMPTY__
# Usage       : always "Public"

# Example rows:
0, 2007_000032, "15 0.981234 104.0 62.0 237.0 334.0 1 0.741234 0.0 80.0 110.0
210.0", Public
1, 2007_000061, __EMPTY__, Public
```

The score threshold for including a detection is **0.05** — this matches the `score_thresh` parameter in both `export_submission_csv()` and `evaluate_map_placeholder()`, so your local val mAP and your Kaggle score are computed on equivalent predictions.

Tip: validate locally first. Before uploading to Kaggle, run `evaluate_map_placeholder()` on your validation set. If it reports a reasonable mAP@50, your model is working correctly and the submission file will be in the right format.

Submission Rules

- **Unlimited submissions** — you may submit as many times as you like before the deadline.
- **Final score** — your highest leaderboard score before the deadline will be used for grading.
- **No test label access** — do not attempt to find or use the test labels. The Kaggle backend evaluates your predictions privately.

Tip: validate locally first. Before uploading to Kaggle, run your inference code on the validation set and compare the output CSV against the `evaluate_map_placeholder()` result from your notebook. If the numbers are close, your CSV format is correct. This saves you a submission slot.

7.2 Part 2 — Deployment

14. **ONNX model file (.onnx)** — exported from your trained PyTorch model as described in Section 3.6.

15. **TensorRT engine (.trt)** — compiled on the Jetson Orin from your ONNX file.
16. **Latency report** — a short script or notebook showing how you measured latency on the Jetson Orin, and the resulting per-image inference time.

7.3 Technical Report (3–5 pages)

Your report must include:

- **Architecture description** — a diagram or table of your StudentNet layers (kernel size, channels, stride, block type)
- **Training curves** — train loss and val loss vs. epoch
- **Part 1 results** — your Kaggle leaderboard mAP@50 and per-class AP from the validation set
- **Part 2 results** — latency on Jetson Orin (TensorRT FP16) vs. the PyTorch baseline on T4
- **Design rationale** — why you chose your architecture, what you tried, what worked and what did not
- **Deployment observations** — any issues encountered during ONNX export or TensorRT conversion, and how you resolved them

8. Grading Rubric

The assignment is graded in two parts. Part 1 accounts for 75% of the total grade; Part 2 accounts for 25%.

Phase	Criterion	Weight	Description
Part 1	mAP@50 vs. Baseline	50%	mAP@50 score from the Kaggle leaderboard. Baseline is the pretrained MobileNetV3-Small (~0.457). No test labels are provided.
Part 1	No Pretrained Weights	15%	All weights trained from scratch. Any use of pretrained weights results in zero for this criterion.
Part 1	Report & Analysis	10%	Architecture design rationale, training curves, per-class AP analysis.
Part 2	ONNX + TensorRT Conversion	15%	Successful export to ONNX and TensorRT engine running on Jetson Orin.
Part 2	Inference Latency on Jetson Orin	10%	End-to-end latency of TensorRT engine on Jetson Orin. Lower is better.

Note on the pretrained weights rule: We will verify compliance by inspecting your notebook. Any call to `weights=...Weights.DEFAULT`, `pretrained=True`, or `load_state_dict` on an external checkpoint will result in a zero for the “No Pretrained Weights” criterion.

9. Academic Integrity

You may discuss high-level design ideas with classmates, but all code must be written independently. Sharing or copying StudentNet implementations is not permitted. You may consult published papers and open-source repositories for architectural inspiration, but you must implement everything yourself and cite any sources you reference in your report.

Good luck, and happy training!
